

Agent Scratch Workspace

A standalone, deep-dive reference for Build 2.5 — every phase, every file, every verification result, in full. Complements (does not replace) the living cheat sheet and the phase diagrams.

STATUS	COMMITTS	CONTINUES	PRECEDES
Complete — all 4 phases	a44da27 → 567a66c	Build 2 (raw-to-queryable pipeline)	Build 3 (pgvector RAG agent)

- What & Why
- Trust Boundary
- Phase 1
- Phase 2
- Phase 3
- Phase 4
- Verification Log
- Known Limitations
- Environment State

What Was Built, and Why

Build 2's pipeline gave the agent something real to read (`clean.allocations`).

Build 2.5 gives it somewhere to **write** — a place to persist a computed result (an aggregation, a derived table) so it can be reused in a later question, without ever touching the boundary that keeps the rest of the database read-only.

This was deliberately scoped as its own mini-build — inserted between Build 2 and Build 3, not folded into either — because it's a distinct agent-architecture capability (agent memory / write-back), not a data-pipeline concern (Build 2) or a retrieval concern (Build 3). See `project_agent-roadmap.md` memory for the full reasoning trail behind that call.

Trust Boundary, Extended

The existing Build 1 boundary is untouched. A second, parallel boundary was added alongside it — not a widening of the first one.

ROLE	CAN DO	CANNOT DO
appdb_reader	SELECT on public, clean, agent_scratch	Any write, anywhere
appdb_agent_writer	CREATE + write in agent_scratch only	Touch public, clean, or raw at all

Why two roles, not one wider role: extending appdb_reader with write access anywhere would mean one credential holds both read-everywhere and write-somewhere — harder to reason about and audit. Two narrow, single-purpose roles keep each one's blast radius small and independently verifiable.

1 Provision the Role and Schema

Goal: create `appdb_agent_writer` and `agent_scratch`, with grants scoped so the writer can touch nothing else, and prove that boundary live before building anything on top of it.

```
006_agent_scratch_role.sql

-- writable role for the agent's scratch workspace. Idempotent: safe to re-run.
DO $$
BEGIN
    IF NOT EXISTS (SELECT FROM pg_catalog.pg_roles WHERE rolname = 'appdb_agent_writer') THEN
        CREATE ROLE appdb_agent_writer WITH LOGIN;
    END IF;
END
$$;

GRANT CONNECT ON DATABASE appdb TO appdb_agent_writer;
```

```
007_agent_scratch_schema.sql

-- the agent's writable scratch schema for persisted findings.
-- Depends on appdb_agent_writer (006) and appdb_reader (003) already existing.

CREATE SCHEMA IF NOT EXISTS agent_scratch;

GRANT USAGE, CREATE ON SCHEMA agent_scratch TO appdb_agent_writer;

GRANT USAGE ON SCHEMA agent_scratch TO appdb_reader;
GRANT SELECT ON ALL TABLES IN SCHEMA agent_scratch TO appdb_reader;
ALTER DEFAULT PRIVILEGES FOR ROLE appdb_agent_writer IN SCHEMA agent_scratch
    GRANT SELECT ON TABLES TO appdb_reader;
```

Why FOR ROLE `appdb_agent_writer` is new here: every earlier default-privilege line (003/004/005) worked without it because the same role (`devuser`) both created the tables and ran the `ALTER DEFAULT PRIVILEGES` statement. Here the table creator (`appdb_agent_writer`) is a different role than the script runner (`devuser`) — the default has to be set explicitly for that other role, or it silently wouldn't apply.

How it was run (PowerShell):

```
Get-Content 006_agent_scratch_role.sql -Raw | docker exec -i pg_local psql -U devuser -d appdb
Get-Content 007_agent_scratch_schema.sql -Raw | docker exec -i pg_local psql -U devuser -d appdb
```

Boundary check — no password needed, via SET ROLE: since `devuser` is effectively a superuser, it can test exactly what `appdb_agent_writer` can and can't do without ever touching its credential.

```
SET ROLE appdb_agent_writer;

-- Both FAILED as expected: permission denied for schema clean / public
CREATE TABLE clean.test_boundary (x int);
CREATE TABLE public.test_boundary (x int);

-- Both SUCCEEDED as expected
CREATE TABLE agent_scratch.test_boundary (id serial primary key, note text);
INSERT INTO agent_scratch.test_boundary (note) VALUES ('boundary check');

RESET ROLE;
-- devuser AND appdb_reader (via a second SET ROLE) could both read the row back
```

2 Build the Guarded Write Tool

Goal: a function the agent can call to persist a result — without ever giving it a "run arbitrary write SQL" tool, and without defaulting to the raw layer's all-TEXT shortcut (which would break real arithmetic on computed results).

Four guardrail layers, none of which alone has to be perfect:

1. The `agent_scratch.` prefix is **hardcoded** in the function — never something the caller can specify.
2. Table/column names are checked against a safe identifier pattern before any SQL is built.
3. SQL is constructed with `psycpg.sql.Identifier`, not string formatting.
4. `appdb_agent_writer` itself is physically incapable of writing anywhere else, regardless of any bug in the three layers above.

Design correction mid-phase: the first draft defaulted every column to TEXT, mirroring `raw.allocations`. The user asked directly whether that would cause problems for calculations — it would have. `raw`'s all-TEXT approach exists to preserve source data before any transformation decision; `agent_scratch` holds *already-computed* results that need real arithmetic and correct sort order. TEXT makes SUM/AVG error outright, and — worse — makes ORDER BY silently sort lexicographically ('9' after '10'), a quiet wrong-answer bug rather than a loud one. Fixed by inferring each column's real Postgres type from the actual Python value passed in.

```
tools.py - additions

import re
from psycpg import sql

SAFE_NAME = re.compile(r"^[a-z_][a-z0-9_]{0,62}$")

# Order matters: bool must come before int, since isinstance(True, int) is True in Python.
PG_TYPE_MAP = {
    bool: "BOOLEAN",
    int: "BIGINT",
    float: "DOUBLE PRECISION",
    str: "TEXT",
    datetime: "TIMESTAMP",
}

def _infer_type(value):
    for py_type, pg_type in PG_TYPE_MAP.items():
        if isinstance(value, py_type):
            return pg_type
    raise TypeError(f"Unsupported value type: {type(value)}")

def save_dataframe(table_name: str, columns: list[str], rows: list[tuple]) -> dict:
    if not SAFE_NAME.match(table_name):
        raise ValueError(f"Invalid table name: {table_name!r}")
    for col in columns:
        if not SAFE_NAME.match(col):
            raise ValueError(f"Invalid column name: {col!r}")
    if not rows:
        raise ValueError("Cannot infer types from an empty result set")

    column_types = [_infer_type(v) for v in rows[0]]

    conn = psycpg.connect(
        host=os.environ.get("POSTGRES_HOST", "127.0.0.1"),
        port=5432,
        dbname=os.environ["POSTGRES_DB"],
        user=os.environ["POSTGRES_AGENT_WRITER_USER"],
        password=os.environ["POSTGRES_AGENT_WRITER_PASSWORD"],
    )

    with conn.cursor() as cur:
        col_defs = sql.SQL(", ").join(
            sql.SQL("{} {}").format(sql.Identifier(col), sql.SQL(pg_type))
            for col, pg_type in zip(columns, column_types)
        )
```

```
create_stmt = sql.SQL("CREATE TABLE IF NOT EXISTS agent_scratch.{ } ({})").format(
    sql.Identifier(table_name), col_defs
)
cur.execute(create_stmt)

insert_stmt = sql.SQL("INSERT INTO agent_scratch.{ } ({}) VALUES ({})").format(
    sql.Identifier(table_name),
    sql.SQL(", ").join(sql.Identifier(c) for c in columns),
    sql.SQL(", ").join(sql.Placeholder() for _ in columns),
)
cur.executemany(insert_stmt, rows)

conn.commit()
conn.close()
return {"schema": "agent_scratch", "table": table_name, "rows_written": len(rows)}
```

Verified live — a mixed-type row (str, int, float, bool) round-tripped with exactly the right Postgres types (confirmed via \d): text, bigint, double precision, boolean. Four escape/injection attempts — a dotted schema-qualified name, a bare SQL keyword, a leading digit, a semicolon-chained DROP — were all rejected by the identifier check before any query touched the database.

3 Wire Into the Agent, Prove the Round Trip

Goal: expose `save_dataframe` as a real tool, and prove a save-then-read-back round trip works across genuinely separate process runs — not just within one session's memory.

```
agent.py - new tool schema

{
  "name": "save_dataframe",
  "description": "Persist a computed result ... to your own scratch workspace ... cannot affect any of the main database tables.",
  "input_schema": {
    "type": "object",
    "properties": {
      "table_name": {"type": "string"},
      "columns": {"type": "array", "items": {"type": "string"}},
      "rows": {"type": "array", "items": {"type": "array"}}
    },
    "required": ["table_name", "columns", "rows"]
  }
}
```

```
agent.py - dispatch addition

elif block.name == "save_dataframe":
    result = save_dataframe(
        block.input["table_name"],
        block.input["columns"],
        [tuple(row) for row in block.input["rows"]],
    )
```

```
tools.py - schema discovery widened again

WHERE table_schema IN ('public', 'clean', 'agent_scratch')
```

Real gap found and fixed mid-phase: run 1 asked the agent to compute an average and save it — worked perfectly, including the same graceful self-correction from Build 2 Phase 3 (tried the table name unqualified, got a real error, retried with the schema prefix on its own). Run 2, a *fresh* process with zero memory of run 1, was asked whether it had a previously saved result — and it flatly said it had no way to check, without even attempting `list_tables`. Root cause: nothing told the model that saved results are discoverable the same way as any other table. Fixed with one line in the system prompt:

Anything you save with `save_dataframe` becomes a normal table you can rediscover later — use `list_tables` and `describe_table` to find and inspect it, the same way you would any other table, before assuming a past result isn't available.

Re-ran after the fix: the agent called `list_tables`, found `agent_scratch.domain_avg_duration`, reasoned it was relevant even though its name didn't literally match the question's wording, inspected it, read it back, and answered correctly with the exact figures from run 1.

4 Automated Boundary Tests

Goal: codify Phase 1's manual SET ROLE boundary check as a real, repeatable pytest suite — the stretch phase, done rather than skipped.

```
test_agent_scratch_boundary.py

def test_writer_has_no_elevated_privileges():
    conn = _connect_as("POSTGRES_USER", "POSTGRES_PASSWORD")
    with conn.cursor() as cur:
        cur.execute("""SELECT rolsuper, rolcreatorole, rolcreatedb
                        FROM pg_roles WHERE rolname = 'appdb_agent_writer'""")
        rolsuper, rolcreatorole, rolcreatedb = cur.fetchone()
    conn.close()
    assert not rolsuper
    assert not rolcreatorole
    assert not rolcreatedb

def test_writer_blocked_from_clean_schema():
    conn = _connect_as("POSTGRES_AGENT_WRITER_USER", "POSTGRES_AGENT_WRITER_PASSWORD")
    with pytest.raises(psycopg.errors.InsufficientPrivilege):
        with conn.cursor() as cur:
            cur.execute("CREATE TABLE clean.pytest_boundary_test (x int)")
    conn.close()

# + test_writer_blocked_from_public_schema (same pattern)
# + test_writer_can_create_and_reader_can_see_it_in_agent_scratch
# (writer creates+inserts, reader connection reads it back, writer drops it)
```

Result: all 4 new tests passed on the first run, alongside the 4 existing test_data_quality.py tests from Build 2 — 8 total, all green, in the same suite.

Verification Log

What was actually exercised against the running system, not just read for plausibility — same standard as every prior build in this project.

Role privilege check

appdb_agent_writer — no Superuser / Creatorole / Createdb attributes.

Boundary check, via SET ROLE

Blocked from clean and public with real permission-denied errors; succeeded in agent_scratch; appdb_reader confirmed able to read what the writer wrote.

Type inference check

A mixed str/int/float/bool row produced exactly text/bigint/double precision/boolean columns, confirmed via \d.

Injection/escape attempts

Four attempts (dotted schema name, bare SQL keyword, leading digit, semicolon-chained DROP) all rejected before touching the database.

Agent round trip, across two separate process runs

Run 1 computed and saved an aggregation; run 2 (fresh process, no shared memory) discovered, read back, and correctly reported the exact same figures.

Full automated suite

9/9 tests passing — 4 from Build 2's test_data_quality.py, 5 from test_agent_scratch_boundary.py (4 from Phase 4, plus the table-cap test added as a same-day follow-up).

Known Limitations & Open Items

- No cleanup policy for scratch tables.

Nothing expires or caps what accumulates in agent_scratch. Deliberately deferred — a manual DROP TABLE is fine for now; automated expiry would be over-engineering for what's currently a single-agent, low-volume workspace.
- Resolved — table-count cap.

MAX_SCRATCH_TABLES = 50 in save_dataframe, checked before any new CREATE TABLE runs (not before every write — appending to an already-existing table is never blocked). Verified live by temporarily lowering the cap to 1: appending to the existing domain_avg_duration table still succeeded while a brand-new table was refused with a clear RuntimeError. Codified as test_scratch_table_cap_blocks_new_tables_but_not_existing_ones (uses monkeypatch to lower the cap to 0 rather than creating 50 real tables). No per-table row/byte-size cap exists yet — not addressed, and lower priority than the table-count gap was.
- Carried forward, still open:

the list_tables/describe_table merge-into-one-call idea (flagged during Build 2 Phase 3) remains unaddressed — now touches three schemas instead of two, same low-priority status.

Environment State

Component	Detail
New role	appdb_agent_writer
New schema	agent_scratch
New credentials	POSTGRES_AGENT_WRITER_USER / POSTGRES_AGENT_WRITER_PASSWORD
New SQL files	006_agent_scratch_role.sql, 007_agent_scratch_schema.sql
New/changed Python	tools.py (+save_dataframe, widened discovery), agent.py (+tool, +dispatch, +prompt line), test_agent_scratch_boundary.py
Live artifact from testing	agent_scratch.domain_avg_duration (kept, real example – not test scaffolding)

Build 2.5 complete. Next: Build 3 — retrieval-augmented agent (pgvector on the existing Postgres). This brief is a point-in-time snapshot of Build 2.5 specifically; ongoing cross-build command reference lives in sql-test-01-cheatsheet.html.